

PHONEBOOK CLI: SISTEM MANAJEMEN BUKU TELEPON BERBASIS TERMINAL

Disusun untuk memenuhi tugas akhir Project Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)



Disusun Oleh:

Muhammad Najmi Khoiri A.	J0403251095
Muhammad Nabil Allatif	J0403251128
Muhammad Tauriq Arya P.	J0403251149

Program Studi Teknologi Rekayasa Perangkat Lunak
Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya proyek dan laporan akhir Project Based Learning (PBL) Mata Kuliah Algoritma dan Struktur Data ini dapat diselesaikan dengan baik.

Proyek yang dikembangkan berupa aplikasi PhoneBook CLI, yaitu sistem manajemen buku telepon berbasis terminal yang memungkinkan pengguna untuk menyimpan, mencari, memperbarui, menghapus, mengurutkan, serta melakukan pencadangan data kontak secara mudah dan terstruktur.

Dalam pengembangan aplikasi ini, kami menerapkan berbagai konsep yang telah dipelajari selama perkuliahan, seperti struktur data Linked List dan Stack, algoritma pencarian dan pengurutan, validasi input, serta file handling menggunakan format CSV.

Kami mengucapkan terima kasih kepada dosen pengampu Mata Kuliah Algoritma dan Struktur Data yang telah memberikan arahan dan bimbingan selama proses pengerjaan proyek ini.

Kami menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan untuk pengembangan yang lebih baik di masa mendatang.

DAFTAR ISI

Bab I	
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Proyek.....	1
1.3 Ruang Lingkup.....	2
Bab II	
ANALISIS DAN PERANCANGAN.....	3
2.1 Deskripsi Sistem.....	3
2.2 Struktur Data yang Digunakan.....	3
2.3 Algoritma yang Digunakan.....	5
2.4 Diagram Alir Program (Flowchart).....	5
2.5 Struktur Program.....	6
Bab III	
IMPLEMENTASI PROGRAM.....	8
3.1 Tampilan Menu Utama.....	8
3.2 Fitur Create.....	8
3.3 Fitur Read.....	9
3.4 Fitur Update.....	9
3.5 Fitur Delete.....	10
3.6 Fitur Searching.....	10
3.7 Fitur Sorting.....	11
3.8 Penyimpanan Data (File Handling).....	12
3.9 Validasi Input dan Error Handling.....	12
Bab IV	
PENGUJIAN DAN ANALISIS.....	14
4.1 Skenario Pengujian.....	14
4.2 Kelebihan Sistem.....	16
4.3 Keterbatasan Sistem.....	16
Bab V	
KENDALA DAN SOLUSI.....	17
Bab VI	
PEMBAGIAN TUGAS ANGGOTA.....	19
Bab VII	
KESIMPULAN DAN SARAN.....	21
Kesimpulan.....	21
Saran.....	21
DAFTAR PUSTAKA.....	22
LAMPIRAN.....	23
Lampiran A. Link Github.....	23
Lampiran B. Screenshot Program.....	23
Lampiran C. Struktur Folder Proyek.....	24
Lampiran D. Commit History Github.....	24

Lampiran E. Source Code Utama.....	25
▶ Import & Dependensi.....	25
▶ Utilitas Tampilan – _clear(), _divider(), _banner(), _menu(), _pause(), _section().....	25
▶ Class PhoneBookApp – __init__(), _load_on_startup(), _save(), _log(), run().....	26
▶ 1. handle_add() – Tambah Kontak Baru.....	27
▶ 2. handle_view() – Tampilkan Semua Kontak (dengan Pagination).....	27
▶ 3. handle_search() – Cari Kontak (Binary Search + Linear Search fallback)....	28
▶ 4. handle_edit() – Edit / Update Kontak.....	29
▶ 5. handle_delete() – Hapus Kontak.....	30
▶ 6. handle_sort() – Urutkan & Tampilkan.....	30
▶ 7. handle_history() – Riwayat Operasi (Stack LIFO).....	31
▶ 8. handle_backup() – Backup / Ekspor Data.....	32
▶ 9. _exit() – Keluar Aplikasi.....	32
▶ _resolve_contact() – Helper Pencarian dengan Partial Fallback.....	32
▶ _display_contacts_paginated() – Helper Tampilan Pagination.....	33
▶ Entry-point – if __name__ == '__main__'.....	34

Bab I

PENDAHULUAN

1.1 Latar Belakang

Dalam kehidupan sehari-hari, pengelolaan data kontak atau buku telepon merupakan kebutuhan yang sangat umum. Seiring bertambahnya jumlah kontak yang dimiliki seseorang, diperlukan sebuah sistem yang mampu mengelola data tersebut secara efisien, terstruktur, dan dapat diakses dengan cepat. Sistem yang baik harus mampu melakukan operasi dasar seperti penambahan, pencarian, pembaruan, dan penghapusan data dengan kinerja yang optimal .

Proyek ini dikembangkan sebagai solusi atas permasalahan tersebut dalam bentuk aplikasi Phonebook CLI (Command-Line Interface) yang dibangun menggunakan bahasa pemrograman Python. Aplikasi ini mengimplementasikan Sorted Linked List sebagai struktur data utama untuk menyimpan kontak secara terurut alfabetis, Stack LIFO untuk mencatat riwayat operasi, serta algoritma Binary Search dan Linear Search untuk pencarian data yang efisien .

Selain itu, aplikasi ini juga menerapkan file handling berbasis CSV agar data kontak dapat tersimpan secara permanen dan tetap tersedia saat program dijalankan kembali. Proyek ini menjadi wadah penerapan nyata dari konsep-konsep yang telah dipelajari dalam mata kuliah Algoritma & Struktur Data, sehingga diharapkan dapat memberikan pemahaman yang lebih mendalam tentang penggunaan struktur data dalam pengembangan aplikasi yang fungsional .

1.2 Tujuan Proyek

Berdasarkan latar belakang masalah tersebut, tujuan dalam proyek ini adalah :

1. Membuat aplikasi manajemen kontak berbasis terminal (CLI) menggunakan Python.
2. Mengimplementasikan Sorted Linked List untuk penyimpanan data kontak secara terurut alfabetis.

3. Mengimplementasikan Stack LIFO untuk mencatat riwayat operasi (operation history).
4. Menerapkan algoritma Binary Search untuk pencarian berdasarkan nama dan Linear Search untuk pencarian berdasarkan nomor telepon dan catatan.
5. Menerapkan konsep file handling dengan format CSV agar data tersimpan secara permanen.
6. Mengembangkan sistem validasi input yang robust untuk mencegah data yang tidak valid masuk ke sistem.
7. Mengembangkan solusi nyata yang dapat digunakan untuk mengelola data kontak sehari-hari.

1.3 Ruang Lingkup

Proyek PhoneBook CLI ini memiliki batasan dan ruang lingkup sebagai berikut:

- Platform: CLI (Console/Terminal) berbasis teks
- Bahasa Pemrograman: Python 3.10+
- Penyimpanan Data: File CSV (phonebook.csv)
- Fitur Utama: CRUD (Create, Read, Update, Delete) kontak
- Binary Search (nama eksak), Linear Search (nomor telepon, catatan, dan pencarian parsial nama)
- Sorting: Insertion sort otomatis saat input data; pengurutan ulang berdasarkan nama (A-Z, Z-A) dan nomor telepon
- Fitur Tambahan: Riwayat operasi (Stack), backup/ekspor data, pagination tampilan

Bab II

ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

PhoneBook CLI adalah aplikasi manajemen buku telepon yang berjalan sepenuhnya di lingkungan terminal (command-line interface). Sistem ini memungkinkan pengguna untuk menambahkan kontak baru, menampilkan seluruh kontak yang tersimpan, mencari kontak berdasarkan nama atau catatan, memperbarui informasi kontak yang sudah ada, menghapus kontak, serta melihat riwayat operasi yang telah dilakukan .

Setiap kontak memiliki tiga atribut utama: nama, nomor telepon, dan catatan (opsional). Data disimpan secara permanen pada file CSV sehingga tetap tersedia ketika program dijalankan kembali. Aplikasi dirancang dengan arsitektur modular yang terdiri dari beberapa file Python yang masing-masing memiliki tanggung jawab yang jelas dan terpisah (separation of concerns) .

Antarmuka pengguna dirancang dengan tampilan yang rapi menggunakan karakter ASCII art untuk membuat border dan tabel. Sistem juga dilengkapi dengan fitur pagination untuk menampilkan kontak dalam jumlah besar, serta validasi input yang ketat untuk memastikan integritas data.

2.2 Struktur Data yang Digunakan

Struktur Data	Implementasi	Fungsi
Sorted Linked List	linked_list.py	Menyimpan data kontak secara terurut alfabetis (A-Z). Setiap node (Contact) terhubung ke node berikutnya melalui pointer next. Operasi insert otomatis mencari posisi yang tepat untuk menjaga urutan.
Stack (LIFO)	stack.py	Menyimpan riwayat operasi yang dilakukan pengguna (ADD, EDIT, DELETE, SEARCH, dll.). Stack

		berbasis linked list dengan kapasitas maksimum 20 item. Ketika penuh, elemen terlama di bagian bawah dibuang otomatis.
Node Contact	contact.py	Setiap objek Contact berfungsi sebagai node dalam Sorted Linked List. Menyimpan data (name, phone, note) sekaligus pointer ke node berikutnya.
Dictionary (dict)	phonebook_app.py	Digunakan sebagai dispatch table (HANDLERS) untuk memetakan pilihan menu (string) ke fungsi handler yang sesuai, sehingga seleksi menu lebih efisien dan mudah diperluas.

Alasan Pemilihan Struktur Data :

Sorted Linked List dipilih sebagai struktur data utama karena kemampuannya menjaga urutan data secara otomatis saat insert (insertion sort in-place), tanpa memerlukan operasi sorting manual yang memakan waktu. Kompleksitas insert adalah $O(n)$ namun data selalu terurut sehingga mendukung Binary Search.

Stack dipilih untuk fitur riwayat operasi karena sifat LIFO (Last In, First Out) yang sesuai dengan kebutuhan menampilkan operasi terbaru di urutan pertama. Dengan Stack berbasis linked list, tidak ada alokasi memori tetap dan ukuran dapat diatur dinamis.

2.3 Algoritma yang Digunakan

Searching :

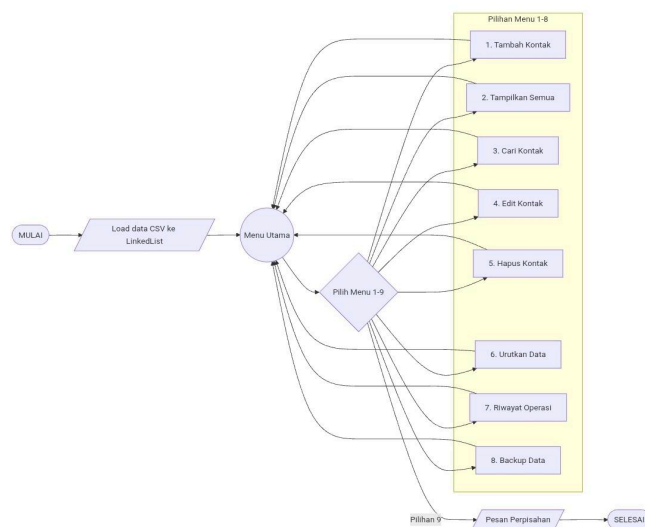
Binary Search digunakan untuk pencarian berdasarkan nama yang eksak (exact match). Karena Sorted Linked List menjaga data tetap terurut A-Z, array hasil `get_all()` dapat langsung digunakan untuk Binary Search dengan kompleksitas $O(\log n)$. Algoritma ini efisien untuk dataset besar karena setiap iterasi mengeliminasi setengah dari ruang pencarian

Linear Search digunakan untuk: (1) pencarian berdasarkan nomor telepon (exact match), (2) pencarian berdasarkan catatan (partial match), dan (3) pencarian parsial nama sebagai fallback ketika Binary Search tidak menemukan hasil. Linear Search memiliki kompleksitas $O(n)$ namun cocok untuk pencarian substring yang tidak mendukung binary search.

Sorting :

Sorted Linked List secara implisit menggunakan algoritma Insertion Sort untuk menjaga urutan saat insert. Setiap kontak baru disisipkan di posisi yang tepat secara alfabetis, sehingga tidak diperlukan sorting manual. Untuk fitur tampilan dengan urutan berbeda (Z-A atau berdasarkan nomor), digunakan Python built-in `sort()` dengan custom key function yang bekerja dengan kompleksitas $O(n \log n)$.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

Nama File	Deskripsi
contact.py	Mendefinisikan class Contact yang berfungsi sebagai node dalam Sorted Linked List. Berisi atribut name, phone, note, dan pointer next. Dilengkapi method to_dict() untuk serialisasi ke CSV, serta operator perbandingan __lt__ dan __eq__ untuk mendukung operasi sorting.
linked_list.py	Mengimplementasikan class LinkedList sebagai Sorted Linked List. Menyediakan operasi insert_sorted(), delete(), search_by_name() (Binary Search), search_by_phone() (Linear Search), search_by_name_partial() (Linear Search parsial), search_by_note(), update(), get_all(), get_sorted(), dan display_all() dengan format tabel.
stack.py	Mengimplementasikan class Stack berbasis linked list (menggunakan class _Node internal) dengan kapasitas maksimum yang dapat dikonfigurasi. Mendukung operasi push(), pop(), peek(), dan to_list(). Secara otomatis membuang elemen terbawah ketika kapasitas penuh.
file_handler.py	Menangani semua operasi I/O terhadap file CSV. Menyediakan fungsi load() untuk membaca data saat startup, save() dengan atomic write (tulis ke .tmp lalu rename), export() untuk menyimpan ke file lain, backup() dengan timestamp otomatis, dan file_info() untuk mendapatkan metadata file.

validator.py	Kumpulan metode statis untuk validasi dan sanitasi input pengguna. Mencakup validate_name() (cek panjang 2-60 karakter, karakter yang diizinkan), validate_phone() (cek format dan panjang 8-15 digit), validate_menu(), dan sanitize() untuk membersihkan karakter berbahaya dari teks bebas.
phoneBookApp.py	File utama yang mengintegrasikan semua komponen. Berisi class PhoneBookApp yang mengelola loop menu utama dan semua handler operasi: handle_add, handle_view (dengan pagination), handle_search, handle_edit, handle_delete, handle_sort, handle_history, dan handle_backup. Juga berisi fungsi utilitas tampilan (_banner, _menu, _divider).
phonebook.csv	File penyimpanan data kontak dalam format CSV dengan kolom name, phone, dan note. Dibuat otomatis saat program pertama kali dijalankan jika belum ada.

Bab III

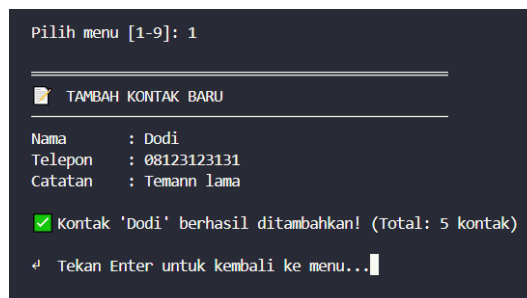
IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama



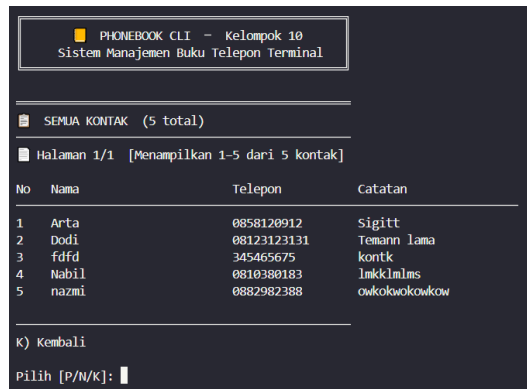
Menu utama menyediakan sembilan fitur utama, yaitu tambah kontak, tampilkan kontak, cari kontak, edit kontak, hapus kontak, sorting, history, backup, dan keluar dari aplikasi.

3.2 Fitur Create



Fitur tambah kontak (menu 1) meminta pengguna untuk memasukkan nama, nomor telepon, dan catatan (opsional). Setiap input divalidasi secara real-time menggunakan class Validator sebelum diterima. Jika nama yang diinputkan sudah ada dalam phonebook, program akan menolak penambahan untuk mencegah duplikasi. Setelah berhasil ditambahkan, kontak langsung disisipkan ke posisi yang tepat dalam Sorted Linked List dan data disimpan ke file CSV.

3.3 Fitur Read



```
PHONEBOOK CLI - Kelompok 10
Sistem Manajemen Buku Telepon Terminal

SEMUA KONTAK (5 total)

Halaman 1/1 [Menampilkan 1-5 dari 5 kontak]

No  Nama          Telepon        Catatan
1   Arta          0858120912    Sigitt
2   Dodi          08123123131   Temann lama
3   fdff          345465675     kontk
4   Nabil         0810380183    lmkkmlms
5   nazmi         0882982388    owkokowkowkow

K) Kembali

Pilih [P/N/K]:
```

Fitur tampilkan kontak (menu 2) menampilkan seluruh kontak yang tersimpan dalam format tabel yang rapi dengan kolom No, Nama, Telepon, dan Catatan. Untuk menangani jumlah kontak yang besar, fitur ini dilengkapi dengan sistem pagination yang menampilkan 10 kontak per halaman. Pengguna dapat bernavigasi menggunakan tombol N (Next/Selanjutnya) dan P (Previous/Sebelumnya).

Kontak ditampilkan dalam urutan A-Z karena data sudah terurut di dalam Sorted Linked List. Jika nama atau nomor telepon terlalu panjang, teks akan dipotong dengan elipsis (...) agar tabel tetap rapi .

3.4 Fitur Update



```
EDIT KONTAK

Nama kontak yang akan diedit: arta

Data saat ini:
No  Nama          Telepon        Catatan
1   Arta          0858120912    Sigitt

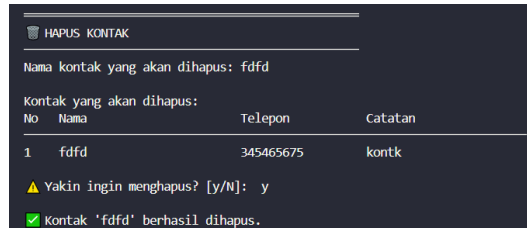
[Tekan Enter untuk melewati / tidak mengubah field]
Nama baru [Arta]: Artanto
Telepon [0858120912]:
Catatan [Sigitt]: rendang

✅ Kontak berhasil diperbarui.
```

Fitur edit kontak (menu 4) memungkinkan pengguna untuk memperbarui salah satu atau semua field kontak (nama, nomor telepon, catatan). Pengguna memasukkan nama kontak yang ingin diedit, kemudian program mencarinya menggunakan Binary Search. Jika tidak ditemukan, program akan menampilkan saran kontak yang mirip melalui pencarian parsial.

Fitur penting dari operasi update adalah penanganan perubahan nama. Ketika nama kontak diubah, sistem secara otomatis menghapus node lama dari Linked List dan menyisipkan kembali node yang sudah diperbarui ke posisi yang tepat, sehingga urutan alfabetis tetap terjaga. Setiap field yang tidak diisi akan mempertahankan nilai lama.

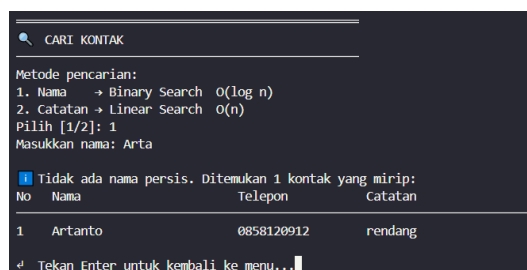
3.5 Fitur Delete



Fitur hapus kontak (menu 5) memungkinkan pengguna untuk menghapus kontak berdasarkan nama. Program menggunakan Binary Search untuk menemukan kontak, kemudian menampilkan detail kontak tersebut beserta konfirmasi sebelum penghapusan dilakukan. Langkah konfirmasi ini mencegah penghapusan data yang tidak disengaja.

Sama seperti fitur edit, jika nama tidak ditemukan secara eksak, program akan mencari kontak yang mirip dan menampilkannya sebagai saran. Setelah kontak dihapus, perubahan langsung disimpan ke file CSV dan dicatat dalam riwayat operasi.

3.6 Fitur Searching



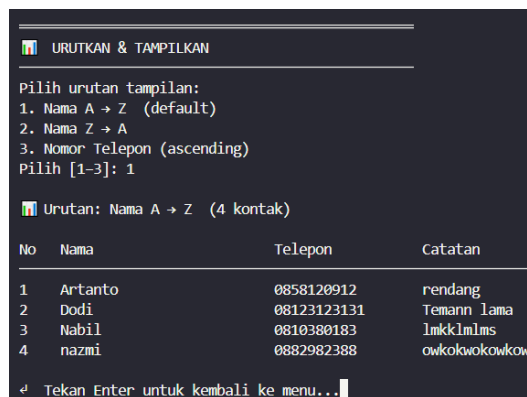
Fitur pencarian (menu 3) menyediakan dua metode pencarian yang dapat dipilih pengguna:

- Pencarian Nama dengan Binary Search: Pengguna memasukkan nama kontak secara eksak. Program mengonversi Linked List ke array dan

melakukan Binary Search untuk menemukan kontak dengan kompleksitas $O(\log n)$. Jika nama tidak ditemukan secara eksak, program secara otomatis melakukan fallback ke Linear Search parsial yang menampilkan semua kontak dengan nama yang mengandung keyword tersebut.

- Pencarian Catatan dengan Linear Search: Pengguna memasukkan kata kunci. Program melakukan traversal seluruh Linked List ($O(n)$) untuk menemukan semua kontak yang catatannya mengandung kata kunci tersebut (case-insensitive, partial match).

3.7 Fitur Sorting



```

URUTKAN & TAMPILKAN

Pilih urutan tampilan:
1. Nama A → Z (default)
2. Nama Z → A
3. Nomor Telepon (ascending)
Pilih [1-3]: 1

Urutan: Nama A → Z (4 kontak)

```

No	Nama	Telepon	Catatan
1	Artanto	0858120912	rendang
2	Dodi	08123123131	Temann lama
3	Nabil	0810380183	lmkk1m1ms
4	nazmi	0882982388	owkokowokow

⌘ Tekan Enter untuk kembali ke menu...

Fitur sorting (menu 6) menyediakan tiga pilihan pengurutan untuk tampilan:

- Nama A ke Z: Menampilkan kontak dalam urutan alfabetis ascending. Karena Sorted Linked List sudah mempertahankan urutan ini, data langsung diambil tanpa sorting ulang (`get_all()`).
- Nama Z ke A: Mengambil semua data lalu mengurutkan secara descending menggunakan Python `sort()` dengan `reverse=True`.
- Nomor Telepon Ascending: Mengurutkan berdasarkan nomor telepon dari yang terkecil ke terbesar menggunakan Python `sort()` dengan `key=lambda c: c.phone`.

Sorting untuk tampilan tidak mengubah struktur internal Sorted Linked List, sehingga urutan A-Z tetap terjaga untuk pencarian Binary Search.

3.8 Penyimpanan Data (File Handling)

Format File:

Data disimpan dalam format CSV (Comma-Separated Values) dengan nama file `phonebook.csv`. File CSV dipilih karena formatnya sederhana, mudah dibaca oleh manusia, dan dapat dibuka dengan aplikasi spreadsheet seperti Microsoft Excel. File memiliki header `name,phone,note` di baris pertama.

```
name,phone,note
Ahmad Fauzi,08123456789,Teman kuliah
Budi Santoso,08234567890,Rekan kerja
Citra Dewi,+6281298765432,Keluarga
```

Cara Membaca File:

Fungsi `load()` pada `FileHandler` membaca file CSV menggunakan `csv.DictReader` yang secara otomatis memetakan setiap baris ke dictionary berdasarkan header. Baris yang tidak memiliki nama atau nomor telepon diabaikan dengan pesan peringatan. Jika file belum ada, `_init_file()` akan membuat file baru dengan header secara otomatis.

Cara Menyimpan File:

Fungsi `save()` menerapkan teknik atomic write: data pertama ditulis ke file sementara (`phonebook.csv.tmp`), kemudian file sementara di-rename menggantikan file asli menggunakan `shutil.move()`. Teknik ini memastikan bahwa jika terjadi kegagalan di tengah proses penulisan (misalnya karena listrik mati), file asli tidak akan rusak atau terhapus.

3.9 Validasi Input dan Error Handling

Sistem validasi input diimplementasikan dalam class `Validator` dengan metode statis yang dapat dipanggil dari mana saja:

Kesalahan Input	Penanganan oleh Sistem
Nama kosong	<code>validate_name()</code> mengembalikan error 'Nama tidak boleh kosong' dan meminta input ulang

Nama terlalu pendek (< 2 karakter)	validate_name() mengembalikan error 'Nama minimal 2 karakter'
Nama terlalu panjang (> 60 karakter)	validate_name() mengembalikan error 'Nama maksimal 60 karakter'
Karakter ilegal pada nama	validate_name() menolak karakter selain huruf, angka, spasi, tanda hubung, apostrof, dan titik
Nomor telepon kosong	validate_phone() mengembalikan error dan meminta input ulang
Nomor telepon terlalu pendek/panjang	validate_phone() memeriksa panjang digit 8-15; di luar range ditolak
Nomor telepon mengandung karakter non-digit	validate_phone() menormalisasi spasi/tanda hubung/kurung, menolak karakter lainnya
Nama kontak sudah ada (duplikat)	handle_add() melakukan cek eksplisit sebelum insert; kontak duplikat ditolak
Pilihan menu di luar range	Loop utama menampilkan pesan error dan meminta input kembali
File CSV tidak ditemukan	FileHandler._init_file() membuat file baru secara otomatis
File CSV rusak/tidak lengkap	load() melewati baris yang tidak valid dengan pesan [SKIP] dan melanjutkan
Input catatan mengandung karakter berbahaya	sanitize() menghapus karakter kontrol ASCII dan mengonversi kutip ganda ke kutip tunggal

Bab IV
PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

Pengujian dilakukan secara manual dengan menjalankan aplikasi dan mencoba berbagai skenario penggunaan. Berikut adalah hasil pengujian yang telah dilakukan:

No	Skenario Pengujian	Input / Kondisi	Hasil yang Diharapkan	Hasil Aktual
1	Tambah kontak baru (valid)	Nama: Ahmad Fauzi, Telepon: 08123456789	Kontak berhasil ditambahkan dan tersimpan di CSV	Berhasil
2	Tambah kontak duplikat	Nama yang sudah ada sebelumnya	Program menolak dengan pesan error duplikat	Berhasil
3	Tambah kontak dengan nama kosong	Nama: (kosong)	Program meminta input ulang	Berhasil
4	Tambah kontak nomor telepon pendek	Telepon: 123 (hanya 3 digit)	Program menolak dan meminta input ulang	Berhasil
5	Tampilkan kontak (list tidak kosong)	Database: 15 kontak	Tampil 10 kontak halaman 1, navigasi ke halaman 2	Berhasil
6	Tampilkan kontak (list kosong)	Database: kosong	Pesan '[Tidak ada kontak]' ditampilkan	Berhasil
7	Cari kontak nama eksak (ada)	Keyword: 'Ahmad Fauzi'	Kontak ditemukan via Binary Search	Berhasil
8	Cari kontak nama eksak (tidak ada)	Keyword: 'Xyz Abc'	Tidak ada hasil, fallback ke partial search	Berhasil

9	Cari kontak nama parsial	Keyword: 'Ahmad'	Menampilkan semua kontak yang namanya mengandung 'Ahmad'	Berhasil
10	Cari kontak berdasarkan catatan	Keyword: 'kuliah'	Semua kontak dengan catatan mengandung 'kuliah' ditampilkan	Berhasil
11	Edit kontak (ubah nama)	Ganti 'Ahmad' menjadi 'Ahmad Baru'	Nama berubah, urutan Linked List tetap alfabetis	Berhasil
12	Edit kontak (ubah telepon saja)	Nomor telepon baru diisi, nama & catatan dikosongkan	Hanya nomor telepon yang berubah	Berhasil
13	Hapus kontak (ada)	Nama kontak yang ada, konfirmasi 'y'	Kontak terhapus, data tersimpan di CSV	Berhasil
14	Hapus kontak (tidak ada)	Nama yang tidak ada di database	Pesan error kontak tidak ditemukan	Berhasil
15	Sorting nama A ke Z	Pilih sorting nama A-Z	Kontak tampilurut A-Z (sesuai Linked List)	Berhasil
16	Sorting nama Z ke A	Pilih sorting nama Z-A	Kontak tampilurut Z-A	Berhasil
17	Sorting nomor telepon	Pilih sorting nomor telepon	Kontak tampilurut berdasarkan nomor	Berhasil
18	Riwayat operasi (Stack)	Setelah beberapa operasi, buka menu history	Operasi terbaru muncul paling atas (LIFO)	Berhasil

19	Backup otomatis	Pilih backup dengan timestamp	File backup_YYYYMMDD_HHMMSS.csv dibuat	Berhasil
20	Persistensi data (restart program)	Tutup program, jalankan kembali	Data kontak tetap ada, dimuat dari CSV	Berhasil

4.2 Kelebihan Sistem

1. Data tersimpan permanen menggunakan file CSV, sehingga tidak hilang saat program ditutup.
2. Sorted Linked List menjamin data selalu terurut A-Z secara otomatis tanpa perlu sorting manual setelah setiap operasi.
3. Binary Search memberikan pencarian yang efisien ($O(\log n)$) untuk dataset besar.
4. Teknik atomic write pada penyimpanan CSV mencegah kerusakan data jika terjadi kegagalan di tengah proses penulisan.
5. Sistem validasi input yang komprehensif mencegah data yang tidak valid masuk ke sistem dan membuat program tidak mudah crash.
6. Arsitektur modular (separation of concerns) membuat kode mudah dipahami, dipelihara, dan dikembangkan.
7. Fitur fallback dari Binary Search ke Linear Search parsial memberikan pengalaman pengguna yang lebih baik ketika nama tidak ditemukan secara eksak.
8. Riwayat operasi dengan Stack memberikan visibilitas terhadap aktivitas yang telah dilakukan selama sesi berjalan.

4.3 Keterbatasan Sistem

1. Masih berbasis CLI (Command-Line Interface) sehingga tampilan kurang menarik dibandingkan aplikasi dengan antarmuka grafis (GUI).
2. Belum mendukung database relasional; penyimpanan CSV tidak efisien untuk dataset yang sangat besar (jutaan kontak).

3. Belum mendukung multiuser; file CSV tidak memiliki mekanisme locking sehingga tidak aman untuk diakses bersamaan oleh lebih dari satu proses.
4. Binary Search memerlukan konversi ke array terlebih dahulu ($O(n)$), sehingga keuntungan $O(\log n)$ berkurang; untuk dataset sangat besar sebaiknya menggunakan struktur data seperti B-Tree atau Hash Table.
5. Tidak ada fitur undo; riwayat operasi hanya untuk informasi dan tidak dapat digunakan untuk membatalkan aksi yang sudah dilakukan.
6. Pencarian hanya mendukung kata kunci tunggal; belum mendukung pencarian multi-kata kunci atau pencarian dengan operator logika (AND, OR).

Bab V

KENDALA DAN SOLUSI

Selama proses pengembangan proyek PhoneBook CLI, kami menghadapi beberapa kendala teknis yang cukup menantang. Berikut adalah kendala-kendala tersebut beserta solusi yang kami terapkan:

Kendala	Solusi yang Diterapkan
Data kontak hilang saat program ditutup	Mengimplementasikan file handling dengan FileHandler yang menyimpan data ke format CSV menggunakan atomic write. Setiap operasi CRUD langsung diikuti dengan pemanggilan <code>save()</code> untuk memastikan data selalu tersimpan.
Program crash saat pengguna memasukkan input yang tidak valid (nomor telepon berupa huruf, nama kosong, dll.)	Membuat class Validator dengan metode <code>validate_name()</code> dan <code>validate_phone()</code> yang menggunakan regex untuk validasi format. Metode <code>prompt_until_valid()</code> memastikan program terus meminta input hingga nilai yang valid diberikan.

Menjaga urutan alfabetis Linked List saat ada operasi update nama	Ketika nama diubah, sistem menghapus node lama dari Linked List terlebih dahulu, memperbarui nama pada objek Contact, kemudian menyisipkan kembali menggunakan <code>insert_sorted()</code> sehingga posisi dalam urutan diperbarui secara otomatis.
Binary Search tidak dapat langsung diterapkan pada Linked List	Mengimplementasikan metode <code>get_all()</code> yang mengkonversi Linked List ke list Python, kemudian menerapkan Binary Search pada array tersebut. Meskipun ada overhead konversi $O(n)$, Linked List tetap dipertahankan untuk menjaga efisiensi insert.
Tampilan kontak yang terlalu panjang merusak format tabel di terminal	Menambahkan logika truncate pada <code>_print_table()</code> yang memotong teks yang melebihi lebar kolom yang ditentukan dan menambahkan karakter elipsis (...) sebagai indikator bahwa teks telah dipotong.
Menampilkan banyak kontak sekaligus membuat terminal tidak dapat di-scroll dengan mudah	Mengimplementasikan sistem pagination pada <code>_display_contacts_paginated()</code> yang menampilkan 10 kontak per halaman dengan navigasi N (next) dan P (previous).
Konflik ketika dua anggota tim mengedit file yang sama secara bersamaan	Menggunakan Git untuk version control. Setiap anggota bekerja pada branch terpisah, kemudian melakukan merge ke branch utama. Konflik merge diselesaikan bersama melalui code review.

Selain kendala teknis di atas, kami juga menghadapi tantangan dalam hal koordinasi tim dan pembagian tugas. Komunikasi yang efektif melalui grup diskusi dan sesi coding bersama (pair programming) menjadi kunci keberhasilan kami dalam menyelesaikan proyek ini tepat waktu.

Bab VI

PEMBAGIAN TUGAS ANGGOTA

Proyek PhoneBook CLI dikerjakan oleh tiga anggota tim dengan pembagian tugas sebagai berikut:

Nama	NIM	Tugas Utama	Detail Kontribusi
Muhammad Najmi Khoiri Almunawwar	J0403251095	Linked List, Stack, PhoneBook App	Mengimplementasikan class LinkedList (linked_list.py) dengan operasi insert_sorted, delete, search_by_name (Binary Search), search_by_phone (Linear Search), search_by_note, update, get_sorted, dan _print_table. Mengimplementasikan class Stack (stack.py) berbasis linked list dengan fitur auto-drop elemen terlama. Pada phoneBookApp.py: handle_add, handle_view (dengan pagination), handle_sort.
Muhammad Nabil Allatif	J0403251128	Contact, PhoneBook App	Mengimplementasikan class Contact (contact.py) sebagai node Sorted Linked List dengan atribut name, phone, note, pointer next, method to_dict(), __lt__, dan __eq__ untuk mendukung serialisasi dan sorting. Pada

			phoneBookApp.py: handle_search (dengan fallback dari Binary ke Linear Search) dan handle_edit (dengan logika re-insert untuk perubahan nama).
Muhammad Tauriq Arya Pradita	J0403251149	Validator, File Handler, PhoneBook App	Mengimplementasikan class Validator (validator.py) dengan validasi nama, nomor telepon, pilihan menu, sanitasi input, dan prompt_until_valid. Mengimplementasikan class FileHandler (file_handler.py) dengan load (membaca CSV dengan error handling), save (atomic write), export, backup (timestamp), dan file_info. Pada phoneBookApp.py: handle_delete, handle_history (dengan pagination), handle_backup, serta UI/UX polish (banner, menu, tampilan).

Catatan Aktivitas GitHub:

Seluruh anggota tim berkontribusi pada repository GitHub kelompok. Commit history dapat dilihat pada Lampiran D untuk melihat kontribusi masing-masing anggota secara detail.

Bab VII

KESIMPULAN DAN SARAN

Kesimpulan

1. Proyek PhoneBook CLI berhasil dikembangkan sebagai aplikasi manajemen kontak berbasis terminal yang fungsional dan stabil. Aplikasi berhasil mengimplementasikan operasi CRUD lengkap dengan persistensi data menggunakan file CSV, sehingga data kontak tetap tersedia setelah program ditutup.
2. Implementasi Sorted Linked List terbukti efektif sebagai struktur data utama untuk menyimpan kontak secara terurut alfabetis. Kombinasi Binary Search ($O(\log n)$) untuk pencarian nama eksak dan Linear Search ($O(n)$) untuk pencarian parsial dan berbasis atribut lain memberikan fleksibilitas pencarian yang optimal.
3. Proyek ini memberikan pemahaman mendalam tentang pentingnya pemilihan struktur data yang tepat dalam pengembangan aplikasi nyata. Stack LIFO untuk riwayat operasi, atomic write untuk keamanan data, dan validasi input yang komprehensif adalah bukti nyata penerapan konsep ilmu komputer dalam konteks praktis.

Saran

1. Pengembangan antarmuka grafis (GUI) menggunakan library Python seperti Tkinter, PyQt, atau aplikasi web berbasis Flask/Django untuk meningkatkan kemudahan penggunaan dan aksesibilitas.
2. Migrasi penyimpanan dari CSV ke database relasional (SQLite atau PostgreSQL) untuk mendukung dataset yang lebih besar, query yang lebih kompleks, dan akses multiuser yang aman.
3. Penambahan fitur import kontak dari format vCard (.vcf) atau sinkronisasi dengan layanan kontak cloud seperti Google Contacts.
4. Implementasi fitur undo/redo yang sesungguhnya menggunakan Command Pattern, bukan sekadar riwayat informasi.
5. Penambahan enkripsi pada file CSV untuk melindungi privasi data kontak pengguna.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Data Structures and Algorithms in Python, Wiley, 2013.
- [3] B. Miller and D. Ranum, Problem Solving with Algorithms and Data Structures Using Python, Franklin, Beedle & Associates, 2011.
- [4] Python Software Foundation, "csv — CSV File Reading and Writing," Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/csv.html>. [Accessed: Jun. 2026].
- [5] Python Software Foundation, "re — Regular expression operations," Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/re.html>. [Accessed: Jun. 2026].
- [6] S. S. Skiena, The Algorithm Design Manual, 3rd ed., Springer, 2020.

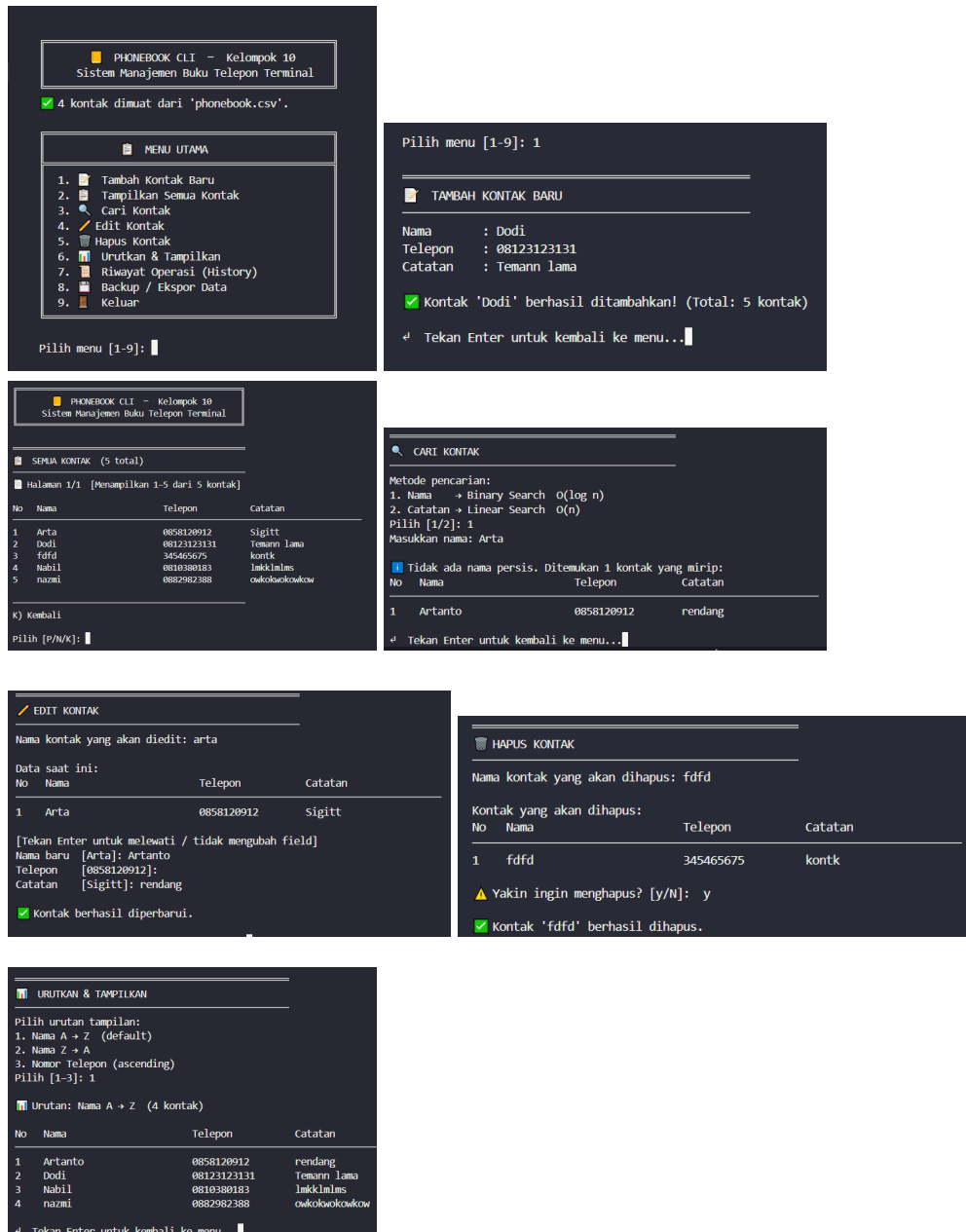
LAMPIRAN

Lampiran A. Link Github

https://github.com/ArtaxSora/ASD-10-DATA_BUKU_TELEPON-PHONEBOOK

=

Lampiran B. Screenshot Program



Lampiran C. Struktur Folder Proyek

phonebook-cli/	
├── phoneBookApp.py	# File utama, entry point aplikasi
├── contact.py	# Class Contact (node Linked List)
├── linked_list.py	# Class LinkedList (Sorted Linked List)
├── stack.py	# Class Stack (LIFO, riwayat operasi)
├── file_handler.py	# Class FileHandler (I/O CSV)
├── validator.py	# Class Validator (validasi & sanitasi input)
├── phonebook.csv	# File penyimpanan data kontak (auto-generated)
└── README.md	# Dokumentasi proyek

Lampiran D. Commit History Github

The screenshot shows the GitHub interface for the repository 'ArtaxSora / ASD-10-DATA_BUKU_TELEPON-PHONEBOOK-'. The 'Commits' tab is selected, showing a list of commits on the 'main' branch. The commits are grouped by date:

- Commits on Jun 23, 2026:**
 - Update Readme.md** (Verified, 1f58423) by nabiusuperjowo, 3 days ago.
 - Add video link to Readme.md** (Verified, 5670432) by Nazmikha, 3 days ago.
- Commits on Jun 22, 2026:**
 - finising 2** (370a8e5) by nabiusuperjowo, 4 days ago.
 - finising** (3d350fe) by nabiusuperjowo, 4 days ago.

The screenshot continues the commit history from May 19, 2026, to May 6, 2026:

- Commits on May 19, 2026:**
 - update riwayat data** (1ac7372) by ArtaxSora, committed on May 19.
 - update serch** (23ef710) by ArtaxSora, committed on May 19.
- Commits on May 13, 2026:**
 - update file_handler** (96df94b) by ArtaxSora, committed on May 13.
 - update** (5e2b8b4) by Nazmikha, committed on May 13.
 - PhonebookApp.py update** (58b8e42) by Nazmikha, committed on May 13.
- Commits on May 6, 2026:**
 - Update Stack** (4619f21) by Nazmikha, committed on May 6.

The screenshot shows the commit history for April 15, 2026:

- Commits on Apr 15, 2026:**
 - update file_handler & Add validator** (d887a8b) by ArtaxSora, committed on Apr 15.
 - Update Linked List** (1f90ae2) by Nazmikha, committed on Apr 15.
 - struktual contact** (e6fe9a7) by nabiusuperjowo, committed on Apr 15.
 - Menambahkan file contact** (6f32fa4) by nabiusuperjowo, committed on Apr 15.
 - Update linked_list** (f2f9f74) by DinoGolzine, committed on Apr 15.
 - Struktur kasar** (4d447eb) by ArtaxSora, committed on Apr 15.

Lampiran E. Source Code Utama

Berikut adalah source code file phoneBookApp.py yang merupakan file utama aplikasi. File ini mengintegrasikan seluruh modul (Contact, LinkedList, Stack, FileHandler, Validator) dan berisi semua logika handler operasi CRUD, pencarian, sorting, riwayat operasi, backup, serta utilitas tampilan CLI.

► Import & Dependensi

```
from __future__ import annotations
import os
import sys
from datetime import datetime
from contact import Contact
from linked_list import LinkedList
from stack import Stack
from file_handler import FileHandler
from validator import Validator
```

► **Utilitas Tampilan** – `_clear()`, `_divider()`, `_banner()`, `_menu()`, `_pause()`, `_section()`

```
# Utilitas tampilan
#
def _clear() -> None:
    os.system("cls" if os.name == "nt" else "clear")

def _divider(char: str = "-", width: int = 48) -> str:
    return " " + char * width

def _banner() -> None:
    print()
    print()
    print("
PHONEBOOK CLI — Kelompok 10
Sistem Manajemen Buku Telepon Terminal")
    print()

def _menu() -> None:
    print()
    print("
MENU UTAMA")
    print()
    print(
1. 📝 Tambah Kontak Baru
2. 📋 Tampilkan Semua Kontak
3. 🔍 Cari Kontak
4. ✏️ Edit Kontak
5. 🗑️ Hapus Kontak
6. 📊 Urutkan & Tampilkan
7. 📖 Riwayat Operasi (History)
8. 💾 Backup / Ekspor Data
9. 🚪 Keluar")
    print()

def _pause() -> None:
    input("\n ↵ Tekan Enter untuk kembali ke menu...")

def _section(title: str) -> None:
    print()
    print(_divider("="))
    print(f" {title}")
    print(_divider("-"))
```

► **Class PhoneBookApp – __init__(), _load_on_startup(), _save(), _log(), run()**

```
class PhoneBookApp:
    """
    Kelas orkestrasi utama: menghubungkan LinkedList, Stack,
    FileHandler, dan Validator menjadi satu aplikasi CLI.
    """

    def __init__(self, csv_file: str = "phonebook.csv"):
        self.phonebook = LinkedList()
        self.history = Stack(max_size=20)
        self.file_handler = FileHandler(filename=csv_file)
        self._load_on_startup()

    # — Startup & shutdown —————
    def _load_on_startup(self) -> None:
        contacts = self.file_handler.load()
        for c in contacts:
            self.phonebook.insert_sorted(c)
        total = len(self.phonebook)
        if total:
            print(f" ✅ {total} kontak dimuat dari '{self.file_handler.filename}'.")
        else:
            print(f" ❗ Tidak ada kontak sebelumnya. Mulai dengan kontak baru!")

    def _save(self) -> None:
        self.file_handler.save(self.phonebook.get_all())

    def _log(self, action: str) -> None:
        ts = datetime.now().strftime("%H:%M:%S")
        self.history.push(f"[{ts}] {action}")

    # — Loop utama —————
    def run(self) -> None:
        _clear()
        _banner()
        self._load_on_startup() # Tampilkan info di bawah banner

    HANDLERS = {
        "1": self.handle_add,
        "2": self.handle_view,
        "3": self.handle_search,
        "4": self.handle_edit,
        "5": self.handle_delete,
        "6": self.handle_sort,
        "7": self.handle_history,
        "8": self.handle_backup,
        "9": self._exit,
    }

    while True:
        _menu()
        try:
            choice = input("\n Pilih menu [1-9]: ").strip()
        except (KeyboardInterrupt, EOFError):
            self._exit()
            return

        if choice in HANDLERS:
            HANDLERS[choice]()
            if choice == "9":
                return
        else:
            print(f" ❌ Pilihan tidak valid. Masukkan angka 1-9.")

    # —————
```

► 1. handle_add() – Tambah Kontak Baru

```
# 1. TAMBAH KONTAK
# Anggota 1
#
def handle_add(self) -> None:
    _section("📝 TAMBAH KONTAK BARU")

    # Nama
    name = Validator.prompt_until_valid(
        " Nama : ", Validator.validate_name
    )
    if self.phonebook.search_by_name(name):
        print(f"❌ Kontak dengan nama '{name}' sudah ada.")
        _pause(); return

    # Telepon
    phone = Validator.prompt_until_valid(
        " Telepon : ", Validator.validate_phone
    )

    # Catatan (opsional, bebas)
    note = Validator.sanitize(input(" Catatan : ").strip())

    contact = Contact(
        name = Validator.sanitize(name),
        phone = phone,
        note = note,
    )
    self.phonebook.insert_sorted(contact)
    self._save()
    self._log(f"ADD → '{name}'")

    print(f"\n✅ Kontak '{name}' berhasil ditambahkan! "
          f"(Total: {len(self.phonebook)} kontak)")
    _pause()

#
```

► 2. handle_view() – Tampilkan Semua Kontak (dengan Pagination)

```
# 2. TAMPILKAN SEMUA KONTAK
# Anggota 1
#
def handle_view(self) -> None:
    self._display_contacts_paginated()

#
```

► 3. handle_search() – Cari Kontak (Binary Search + Linear Search fallback)

```
# 3. CARI KONTAK
# Anggota 2
#
def handle_search(self) -> None:
    _section("🔍 CARI KONTAK")
    print(" Metode pencarian:")
    print(" 1. Nama          → Binary Search  O(log n)")
    print(" 2. Catatan → Linear Search  O(n)")
    method = input(" Pilih [1/2]: ").strip()

    if method == "1":
        keyword = input(" Masukkan nama: ").strip()
        if not keyword:
            print(" ❌ Keyword tidak boleh kosong.")
            _pause(); return

        # Coba exact Binary Search lebih dulu
        found = self.phonebook.search_by_name(keyword)
        if found:
            print(f"\n ✅ Ditemukan (Binary Search - exact match):")
            self.phonebook._print_table([found])
            self._log(f"SEARCH → nama='{keyword}' (ditemukan)")
        else:
            # Fallback: partial linear search
            suggestions = self.phonebook.search_by_name_partial(keyword)
            if suggestions:
                print(f"\n ⓘ Tidak ada nama persis. "
                    f"Ditemukan {len(suggestions)} kontak yang mirip:")
                self.phonebook._print_table(suggestions)
                self._log(f"SEARCH → nama='{keyword}' ({len(suggestions)}"
                    mirip)")
            else:
                print(f"\n ❌ Tidak ada kontak dengan nama '{keyword}'.")
                self._log(f"SEARCH → nama='{keyword}' (tidak ditemukan)")

    elif method == "2":
        keyword = input(" Masukkan catatan: ").strip()
        if not keyword:
            print(" ❌ Keyword catatan tidak boleh kosong.")
            _pause(); return

        found = self.phonebook.search_by_note(keyword)
        if found:
            print(f"\n ✅ Ditemukan {len(found)} kontak dengan catatan yang"
                sesuai:")
            self.phonebook._print_table(found)
            self._log(f"SEARCH → catatan='{keyword}' ({len(found)} ditemukan)")
        else:
            print(f"\n ❌ Tidak ada kontak dengan catatan '{keyword}'.")
            self._log(f"SEARCH → catatan='{keyword}' (tidak ditemukan)")
        else:
            print(" ❌ Pilihan tidak valid.")

    _pause()

#
```


► 4. handle_edit() – Edit / Update Kontak

```
# 4. EDIT KONTAK
# Anggota 2
#
def handle_edit(self) -> None:
    _section("✂ EDIT KONTAK")

    name = input(" Nama kontak yang akan diedit: ").strip()
    contact = self._resolve_contact(name)
    if contact is None:
        _pause(); return

    print(f"\n Data saat ini:")
    self.phonebook._print_table([contact])
    print("\n [Tekan Enter untuk melewati / tidak mengubah field]")

    # — Nama baru —
    new_name_raw = input(f" Nama baru [{contact.name}]: ").strip()
    new_name: str | None = None
    if new_name_raw:
        ok, msg = Validator.validate_name(new_name_raw)
        if not ok:
            print(f" ✖ {msg}"); _pause(); return
        new_name = Validator.sanitize(new_name_raw)

    # — Telepon baru —
    new_phone_raw = input(f" Telepon [{contact.phone}]: ").strip()
    new_phone: str | None = None
    if new_phone_raw:
        ok, result = Validator.validate_phone(new_phone_raw)
        if not ok:
            print(f" ✖ {result}"); _pause(); return
        new_phone = result

    # — Catatan baru —
    new_note_raw = input(f" Catatan [{contact.note}]: ").strip()
    new_note: str | None = Validator.sanitize(new_note_raw) if new_note_raw else
None

    # Gunakan contact.name (nama yang sudah diverifikasi) bukan input awal
    success, msg = self.phonebook.update(
        contact.name,
        new_name = new_name,
        new_phone = new_phone,
        new_note = new_note,
    )

    if success:
        self._save()
        final = new_name if new_name else contact.name
        self._log(f"EDIT → '{contact.name}' → '{final}'")
        print(f"\n ✔ {msg}")
    else:
        print(f"\n ✖ {msg}")

    _pause()

#
```

► 5. handle_delete() – Hapus Kontak

```
# 5. HAPUS KONTAK
# Anggota 3
#
def handle_delete(self) -> None:
    _section("🗑️ HAPUS KONTAK")

    name = input(" Nama kontak yang akan dihapus: ").strip()
    contact = self._resolve_contact(name)
    if contact is None:
        _pause(); return

    print(f"\n Kontak yang akan dihapus:")
    self.phonebook._print_table([contact])

    confirm = input("\n ⚠️ Yakin ingin menghapus? [y/N]: ").strip().lower()
    if confirm == "y":
        # Gunakan contact.name (nama yang sudah diverifikasi)
        deleted = self.phonebook.delete(contact.name)
        if deleted:
            self._save()
            self._log(f"DELETE → '{contact.name}'")
            print(f"\n ✅ Kontak '{contact.name}' berhasil dihapus.")
        else:
            print(f"\n ❌ Gagal menghapus kontak.")
    else:
        print(" ℹ️ Penghapusan dibatalkan.")

    _pause()

#
```

► 6. handle_sort() – Urutkan & Tampilkan

```
# 6. SORT & TAMPILKAN
# Anggota 1
#
def handle_sort(self) -> None:
    _section("📊 URUTKAN & TAMPILKAN")
    print(" Pilih urutan tampilan:")
    print(" 1. Nama A → Z (default)")
    print(" 2. Nama Z → A")
    print(" 3. Nomor Telepon (ascending)")
    choice = input(" Pilih [1-3]: ").strip()

    if choice == "1":
        data = self.phonebook.get_sorted(reverse=False, by="name")
        label = "Nama A → Z"
    elif choice == "2":
        data = self.phonebook.get_sorted(reverse=True, by="name")
        label = "Nama Z → A"
    elif choice == "3":
        data = self.phonebook.get_sorted(reverse=False, by="phone")
        label = "Nomor Telepon ↑"
    else:
        print(" ❌ Pilihan tidak valid.")
        _pause(); return

    print(f"\n 📊 Urutan: {label} ({len(data)} kontak)\n")
    self.phonebook._print_table(data)
    _pause()

#
```

► 7. handle_history() – Riwayat Operasi (Stack LIFO)

```
# 7. RIWAYAT OPERASI
# Anggota 3
#
def handle_history(self) -> None:
    items = self.history.to_list()
    total = len(items)

    if not items:
        _section("📁 RIWAYAT OPERASI")
        print(" [Belum ada riwayat operasi]")
        _pause()
        return

    page = 0
    items_per_page = 5
    total_pages = (total + items_per_page - 1) // items_per_page

    while True:
        _clear()
        _banner()
        _section(f"📁 RIWAYAT OPERASI  ({total}/20 slot terisi)")

        # Hitung range item untuk halaman saat ini
        start_idx = page * items_per_page
        end_idx = min((page + 1) * items_per_page, total)

        print(f" 📄 Halaman {page + 1}/{total_pages}\n")

        # Tampilkan 5 item
        for i in range(start_idx, end_idx):
            display_num = i + 1
            mark = " ← terbaru" if i == 0 else ""
            print(f" {display_num:2}. {items[i]}{mark}")

        print()
        print(_divider("-"))

        # Menu navigasi
        menu_options = []
        if page > 0:
            menu_options.append("P) Sebelumnya")
        if page < total_pages - 1:
            menu_options.append("N) Selanjutnya")
        menu_options.append("K) Kembali ke Menu")

        print(" " + " | ".join(menu_options))

        choice = input("\n Pilih [P/N/K]: ").strip().upper()

        if choice == "K":
            break
        elif choice == "P" and page > 0:
            page -= 1
        elif choice == "N" and page < total_pages - 1:
            page += 1
        else:
            print(" ❌ Pilihan tidak valid.")
            input(" ↵ Tekan Enter...")

#
```

► 8. handle_backup() – Backup / Ekspor Data

```
# 8. BACKUP / EKSPOR
# Anggota 3
#
def handle_backup(self) -> None:
    _section("📁 BACKUP / EKSPOR DATA")
    print(" 1. Buat backup otomatis (timestamp)")
    print(" 2. Ekspor ke nama file kustom")
    choice = input(" Pilih [1/2]: ").strip()

    if choice == "1":
        self.file_handler.backup()
        self._log("BACKUP → backup otomatis")
    elif choice == "2":
        filename = input(" Nama file tujuan (contoh: backup.csv): ").strip()
        if not filename.endswith(".csv"):
            filename += ".csv"
        self.file_handler.export(filename)
        self._log(f"EXPORT → '{filename}'")
    else:
        print(" ❌ Pilihan tidak valid.")

    _pause()

#
```

► 9. _exit() – Keluar Aplikasi

```
# 9. KELUAR
#
def _exit(self) -> None:
    print()
    print(" 🙌 Terima kasih telah menggunakan PhoneBook CLI!")
    info = self.file_handler.file_info()
    if info.get("exists"):
        print(f" 📁 Data tersimpan di: {info['path']}")
    print(" Sampai jumpa!\n")

#
```

► _resolve_contact() – Helper Pencarian dengan Partial Fallback

```
# HELPER PRIVAT: resolve kontak dengan fallback partial
#
def _resolve_contact(self, name: str) -> Contact | None:
    """
    Cari kontak dengan nama eksak.
    Jika tidak ada, tampilkan saran partial dan minta nama ulang.
    Return Contact atau None.
    """
    contact = self.phonebook.search_by_name(name)
    if contact:
        return contact

    suggestions = self.phonebook.search_by_name_partial(name)
    if suggestions:
        print(f"\n ⓘ '{name}' tidak ditemukan persis. "
              f"Kontak yang mirip:")
        self.phonebook._print_table(suggestions)
        exact = input("\n Masukkan nama lengkap (atau Enter untuk batal): ")
        exact = exact.strip()
        if not exact:
            return None
        contact = self.phonebook.search_by_name(exact)
        if contact:
            return contact
```

```
print(f" ❌ Kontak '{name}' tidak ditemukan.")
return None
```

```
# _____
```

► `_display_contacts_paginated()` – Helper Tampilan Pagination

```
# HELPER PRIVAT: Tampilkan kontak dengan pagination
# _____
def _display_contacts_paginated(self) -> None:
    """
    Tampilkan semua kontak dengan pagination (10 data per halaman).
    Mendukung navigasi next, prev, dan kembali ke menu.
    """
    contacts = self.phonebook.get_all()
    total = len(contacts)

    if not contacts:
        _section("📁 SEMUA KONTAK")
        print(" [Tidak ada kontak]")
        _pause()
        return

    items_per_page = 10
    total_pages = (total + items_per_page - 1) // items_per_page
    page = 0

    while True:
        _clear()
        _banner()
        _section(f"📁 SEMUA KONTAK ({total} total)")

        # Hitung range kontak untuk halaman saat ini
        start_idx = page * items_per_page
        end_idx = min((page + 1) * items_per_page, total)
        page_contacts = contacts[start_idx:end_idx]

        print(f" 📄 Halaman {page + 1}/{total_pages} "
              f"[Menampilkan {start_idx + 1}-{end_idx} dari {total} kontak]\n")

        # Tampilkan tabel kontak untuk halaman ini
        # Nomor urut dimulai dari: (page * items_per_page) + 1
        start_num = (page * items_per_page) + 1
        self.phonebook._print_table(page_contacts, start_num=start_num)

        print()
        print(_divider("-"))

        # Menu navigasi
        menu_options = []
        if page > 0:
            menu_options.append("P) Sebelumnya")
        if page < total_pages - 1:
            menu_options.append("N) Selanjutnya")
        menu_options.append("K) Kembali")

        print(" " + " | ".join(menu_options))

        choice = input("\n Pilih [P/N/K]: ").strip().upper()

        if choice == "K":
            break
        elif choice == "P" and page > 0:
            page -= 1
        elif choice == "N" and page < total_pages - 1:
            page += 1
        else:
            if choice not in ["P", "N", "K"]:
                print(" ❌ Pilihan tidak valid.")
            input(" ⌨ Tekan Enter...")
```

► **Entry-point – if __name__ == '__main__'**

```
# Entry-point
# =====
if __name__ == "__main__":
    csv_file = sys.argv[1] if len(sys.argv) > 1 else "phonebook.csv"
    app = PhoneBookApp(csv_file=csv_file)
    app.run()
```

Source code file-file modul pendukung (contact.py, linked_list.py, stack.py, file_handler.py, validator.py) tersedia lengkap di repository GitHub kelompok yang tertera pada Lampiran A.